Daniel Gorin

ST10438307

PROG7311

POE PART 1

2026

Body Wordcount = 1488

ACADEMIC HONESTY DECLARATION:

	Signed
I have read the assessment rules provided in this declaration.	DG
This assessment is my own work.	DG
I have not copied any other student's work in this assessment.	DG
I have not uploaded the assessment question to any website or App offering assessment assistance.	DG
I have not downloaded my assessment response from a website.	DG
I have not used any AI tool without reviewing, re-writing, and re-working this information.	DG
I have not shared this assessment with any other student.	DG
I have not presented the work of published sources as my own work.	DG
I have correctly cited all my sources of information.	DG
My referencing is technically correct, consistent, and congruent.	DG
I have acted in an academically honest way in this assessment	DG

Name: Daniel Gorin

Student Number: ST10438307

Date: 2026/03/27

Contents:

Contents: 1
Summary and Introduction: 2
SECTION 1 — Executive Summary & Introduction 3
Enterprise Framework Strategy: 4
Framework Application: 6
Design Patterns Selection and Justification: 7
Abstract Factory – Creational: 8
Observer – Behavioural: 8
Decorator – Structural: 9
Design Patterns: 9
Abstract Factory: 10
Observer: 10
Decorator: 10
UML Class Diagram: 11
Scalability and Non-Functional 12
Requirements: 13
References: 14

Introduction:

This report outlines the architectural foundation of the GLMS for TechMove's Board of Directors. TechMove currently operates on a legacy system of disconnected spreadsheets, manual phone calls and unstructured emails. This has resulted in fragmented data across departments, lost invoices with no audit trail, compliance failures caused by untracked expired contracts and workflow bottlenecks. The GLMS is designed to replace this with a centralised contract management hub with automated lifecycle tracking, service request processing with enforced business rules, and live international currency conversion.

The architectural approach is enterprise-grade, scalable and containerised. TOGAF's Architectural Development Method will provide the structure to navigate from the legacy to target system. Industry standard design patterns ensure the codebase is maintainable and extensible. The system is designed for deployment on Azure infrastructure with horizontal scaling, load balancing and caching, ensuring it can meet TechMove's current and future needs.

Enterprise Framework Strategy:

An enterprise architecture framework structures decision making, aligning technology, processes and information with business goals (Raza, 2020).

TOGAF structures the full design, planning and governance of an enterprise system through an exact order of tasks from problem statement to deployed solution (Raza, 2020). TOGAF originated in the US DoD in 1995 and has since evolved into an industry-standard framework widely used by Fortune 500 companies (Priatna, 2025). The modern incarnation supports modularity and is agile compatible remaining relevant with modern industry practices (Melchior, 2025).

The Architectural Development Method (ADM) is fundamental to TOGAF, it is a cycle of phases with clearly defined inputs, outputs and progress triggers (Gupta, 2025). Each phase informs the next, with phases revisited as new information is uncovered.

Phases A-D can be Brocken down as follows:

A. Defines the scope and obtains official sign off
B. Determines current and goal infrastructure and analyses the gap between
C. The Design of data and application layers
D. The Design of technology infrastructure

The Remaining phases handle delivery, implementation and monitoring.

The ADM structures the transformation from current to goal state, directly applicable to TechMove's legacy migration (Priatna, 2025). TOGAFs thorough documentation provides a clear paper trail ensuring compliance. TOGAF’s domain by domain phase structure supports the breadth of the project covering business workflows, data, application and infrastructure while ensuring coherence throughout (Gupta, 2025).

Framework Application:

TOGAF ADM Applied to TechMove GLMS

ADM Phase	Focus Areas	Documents Generated	Techmove Application
A	• Define Scope • Identify Stakeholders	• **Architecture Vision** • **Statement of Architecture Work** • **Stakeholder Register**	• **Vision agreed:** replace the fragmented legacy system (spreadsheets, emails, manual phone calls) with a centralised GLMS • **Scope confirmed:** as contract management, service request processing, international currency conversion, and containerised deployment
B	• **Investigate Current State** • **Determine Goal State** • **Analyse Gap between current and goal**	• **Architecture Definition** • **Gap Analysis** • **Business Capability Map**	• **Current state:** contracts managed in spreadsheets, service requests raised by phone, invoices lost, no enforcement of contract status rules, compliance failures around expired contracts • **Target state:** automated Draft→Active→Expired→On Hold contract lifecycle, ServiceRequests blocked against Expired or On Hold contracts, digitally auditable invoice trail, SLA tracking per client
C	• Data Later Desing • Application Layer Design	• Architecture Definition (updated adding data and applicaiton architecture) • Data and Application Architecture Requirements	• **Data entities:** Client, Contract, ServiceRequest • **Applications:** ASP.NET Core MVC frontend, Cloud SQL Databse, Cloud PDF storage for documents, Live Currency Exchange.
D	• Infrastructure Desing (deployment networking, scaling and security)	• Architecture Definition (updated adding technology architecture) • Technology Architecture Requirements	• **Infrastructure:** Azure App Service, Azure SQl Databse, Azure Filestorage, Currency Exchnage API (ExchangeRate-API, CurrencyLayer or Open Exchange Rates)

Design Patterns Selection + Justification:

Design patterns are proven, reusable blueprints for solving recurring problems in object oriented software development. They encourage modularity and readability (Pankaj & Manikandan, 2025).

Abstract Factory - Creational:

Abstract factory provides an interface for creating related objects without specifying their exact class. The caller asks for an object the factory decides which concrete type to produce (Refactoring Guru, n.d.-a).

The GLMS involves standard, SLA and international contracts with each needing to be handled differently. Hardcoding these differences into the controller would make future changes increasingly complicated. The abstract factory pattern solves this issue by defining an IContractFactory interface with a CreateContract() method, the controller calls the interface and the factory handles contract types keeping the controller clean and centralising contract creation logic.

Observer - Behavioural:

The observer defines a one-to-many relationship between objects, when an object change state the dependant objects are notified (Poorte, 2024).

service requests on the GLMS should be blocked if the parent contract is expired or on hold. The controller action that creates the service request could manually check this, but that would bulk up the controller and spread validation logic across the project. The Observer pattern handles this, making the contract the subject when Contract.Status changes to “Expired” or “On Hold”, all observers will be automatically notified including the ServiceRequestValidator which would comply with the core business rules and block new requests for that contract. This effectively separates the Contract and service request logic making the code readable and easy to edit.

Decorator - Structural:

A Decorator adds additional layers to an object adding behaviours without modifying the original object (Refactoring Guru, n.d.-b).

The GLMS requires uploaded documents to be validated for file type embedding this in the upload service couples two concerns and complicates future changes. The Decorator solves this issue by wrapping IFileUploadService in PdfValidationDecorator that checks the file type before passing it on to the inner storage service. This keeps the storage service purely focused on storage and makes changing or adding validation rules simple by adding additional decorator layers.

Design Patterns Application:

Abstract Factory:

The Abstract Factory is built around the IContractFactory interface which declares a single CreateContract() method returning a Contract object. There are three concrete classes implement this interface StandardContractFactory, SLAContractFactory, and InternationalContractFactory, each encapsulating the creation and configuration logic for that contract type. The ContractService declares IContractFactory as a constructor parameter it does not reference a concrete factory class directly. Rather than creating a factory itself, the ContractService receives one through its constructor at runtime, this is dependency injection, where ASP.NET Core handles providing the correct implementation automatically. Changing contract type requires no changes to the controller or service.

Observer:

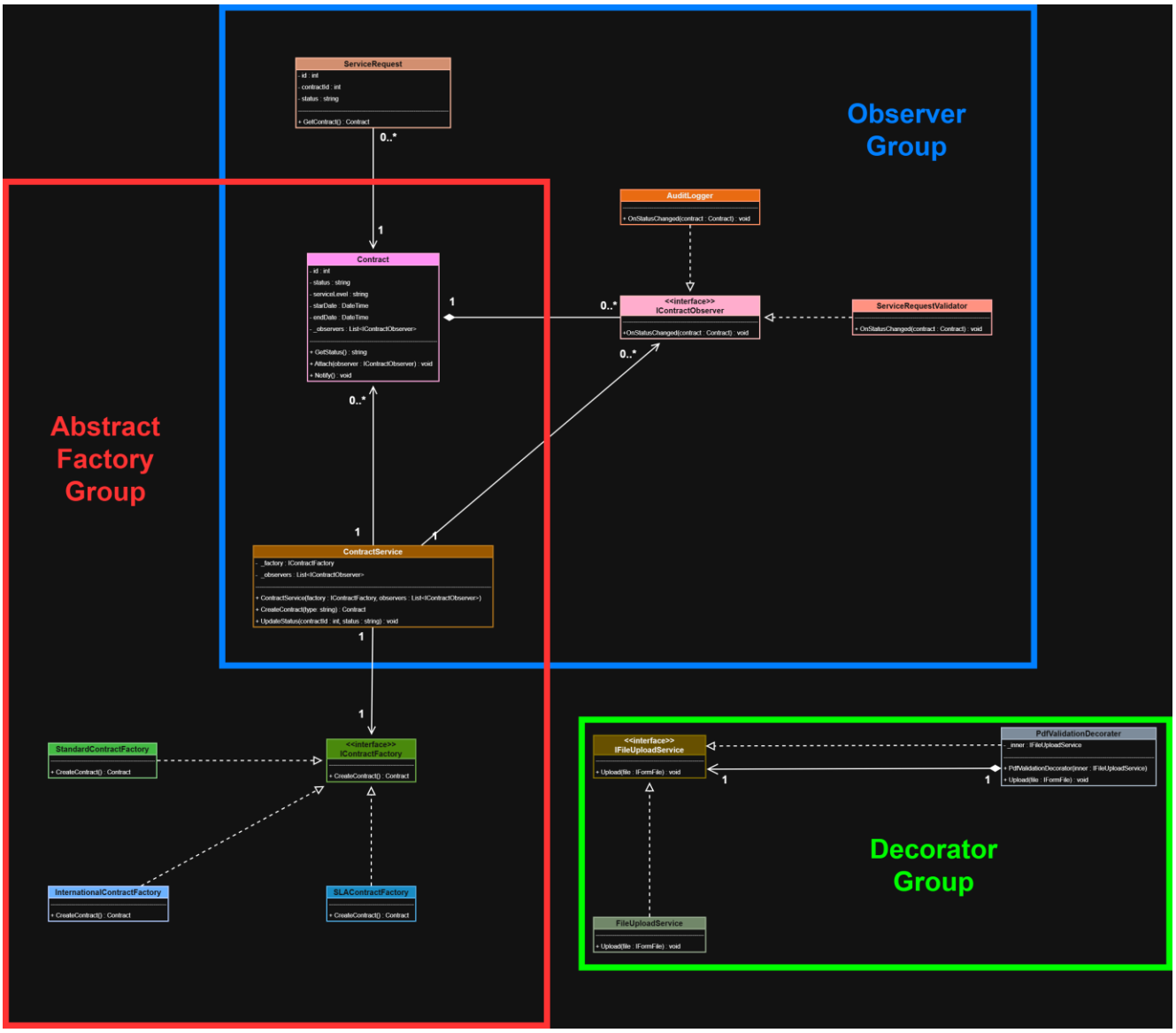
The Observer pattern is built around the IContractObserver interface, which any observer class must implement. The Contract class acts as the subject it keeps a list of registered observers and when its status changes it notifies each registered observer. There are two concrete observer classes that implement this the ServiceRequestValidator, which enforces the core business rule by blocking new service requests when the contract is Expired or On Hold, and the AuditLogger, which records the status change for the compliance trail. When ContractService.UpdateStatus() is called the contract fires off its notifications automatically. The observers themselves are passed into the ContractService through its constructor, meaning they can be changed wihtout impacting the service itself.

Decorator:

The Decorator is built around the IFileUploadService interface which declares an Upload(IFormFile file) method. The FileUploadService handles the actual file storage operation. PdfValidationDecorator holds a private reference to an IFileUploadService instance its inner service, passed in through its constructor. When Upload() is called on the decorator, it first checks the file type, throwing a ValidationException if the file is not a

PDF, before delegating to the inner service. Additional validation layers can be stacked as further decorators without any changes to existing classes.

UML Class Diagram:



Scalability + Non-Functional Requirements:

Interoperability is a non-functional requirement that describes how easily a system can share information and communicate (integrate) with other systems and external hardware (Alexandrian, n.d.). The GLMS already demonstrates this through the exchange rate API integration, with further integrations expected as TechMove expands OpenAPI 3.0 is a standard specification for describing RESTful APIs that allows both developers and external systems to understand and interact with a service's capabilities without needing access to its source code (Swagger, n.d.). Applying this to all external facing endpoints is a logical and clearly defined target for the GLMS's interoperability.

Availability is a core non-functional requirement it describes the consistency with which a system is operational and accessible. It is measured as percentage of uptime. The GLMS is a business-critical enterprise system, an uptime of 99.9% is an industry accepted goal for such a system (Webalert, 2026). Contracts and service requests are time sensitive with downtime leading to compliance and workflow failures.

Scalability is the ability of a system to handle growing demand without degradation in performance or availability (Microsoft Azure, n.d.). If TechMove acquires a competitor and 50000 new contracts enter the GLMS overnight, the Azure SQL Database would face a surge in read and write operations while the ASP.NET Core API layer handles a spike in concurrent requests. This would lead to response times degrading, requests timing out, and service request submissions beginning to fail. Users would face failed submissions, stale data and SLA breaches and compliance failures would occur. The GLMS scalability strategy must address this through horizontal scaling, load balancing and caching.

Vertical scaling adds resources to a single server while horizontal scaling adds additional parallel servers. (Microsoft Learn, n.d.). The Azure App Service supports automatic horizontal scaling when the 50000 contract load spike occurs, additional calls to the GLMS backend are provisioned for automatically and scaled back down when demand stabilises, with no downtime or impact on the frontend or database.

Load balancing is the process of distributing the workload across multiple servers, so that no one server is overwhelmed (CloudFlare, n.d.). The Azure Application Gateway is a web traffic load balancer hat distributes calls across all running API instances, ensuring no single instance is overwhelmed during the 50000 contract surge (Microsoft Learn, 2025).

Caching is storing frequently accessed data in temporary highspeed memory so that it can be retrieved quickly without causing additional strain on the system (Redis, n.d.). Azure Managed Redis is a fully managed caching service that integrates directly with the rest of our azure infrastructure and would be applicable to the GLMS (Microsoft Azure, n.d.). In the 50000 contract scenario, every ServiceRequest creation triggers a contract status check, caching contract status data in Azure Managed Redis means the database is not queried on every request, significantly reducing query volume.

References:

Alexandrian, R. (n.d.). *Non-Functional Requirements (Interoperability):* Argon Digital. Retrieved March 23, 2026, from https://argondigital.com/blog/product-management/non-functional-requirements-interoperability-requirements/

CloudFlare. (n.d.). *What is load balancing? | How load balancers work*. CloudFlare. Retrieved March 24, 2026, from https://www.cloudflare.com/learning/performance/what-is-load-balancing/

Gupta, A. (2025, December 5). *Overview of TOGAF ADM Phases*. StarAgile. https://staragile.com/blog/togaf-adm-phases

Melchior, S. (2025, August 4). *TOGAF or Zachman?* Boldo. https://www.boldo.io/en/wiki/togaf-or-zachman

Microsoft Azure. (n.d.). *Scaling up vs. scaling out An intro to database scalability in cloud computing.* Microsoft Azure. Retrieved March 24, 2026, from https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/scaling-out-vs-scaling-up

Microsoft Azure. (n.d.). *Azure Managed Redis*. Microsoft Azure. Retrieved March 26, 2026, from https://azure.microsoft.com/en-us/products/managed-redis

Microsoft Learn. (2025, December 17). *What is Azure Application Gateway?* Microsoft Learn. https://learn.microsoft.com/en-us/azure/application-gateway/overview

Pankaj, K., & Manikandan, K. (2025, June 13). *Gang of 4 Design Patterns Explained: Creational, Structural, and Behavioral*. DigitalOcean. https://www.digitalocean.com/community/tutorials/gangs-of-four-gof-design-patterns

Poorte, J. (2024, March 17). *C# Observer Design Pattern*. Dofactory. https://www.dofactory.com/net/observer-design-pattern

Priatna, G. G. (2025, December 1). *Enterprise Architecture Frameworks: TOGAF vs. Zachman Explained*. Gun Gun. https://gungunpriatna.com/blog/enterprise-architecture-frameworks-togaf-vs-zachman-explained

Raza, M. (2020, March 27). *TOGAF vs Zachman: What's The Difference?* The Business of IT Blog. https://www.bmc.com/blogs/togaf-vs-zachman/

Redis. (n.d.). *Your high-performance caching solution*. Redis. Retrieved March 25, 2026, from https://redis.io/solutions/caching/

Refactoring Guru. (n.d.-a). *Abstract Factory*. Refactoring Guru. Retrieved March 20, 2026, from https://refactoring.guru/design-patterns/abstract-factory

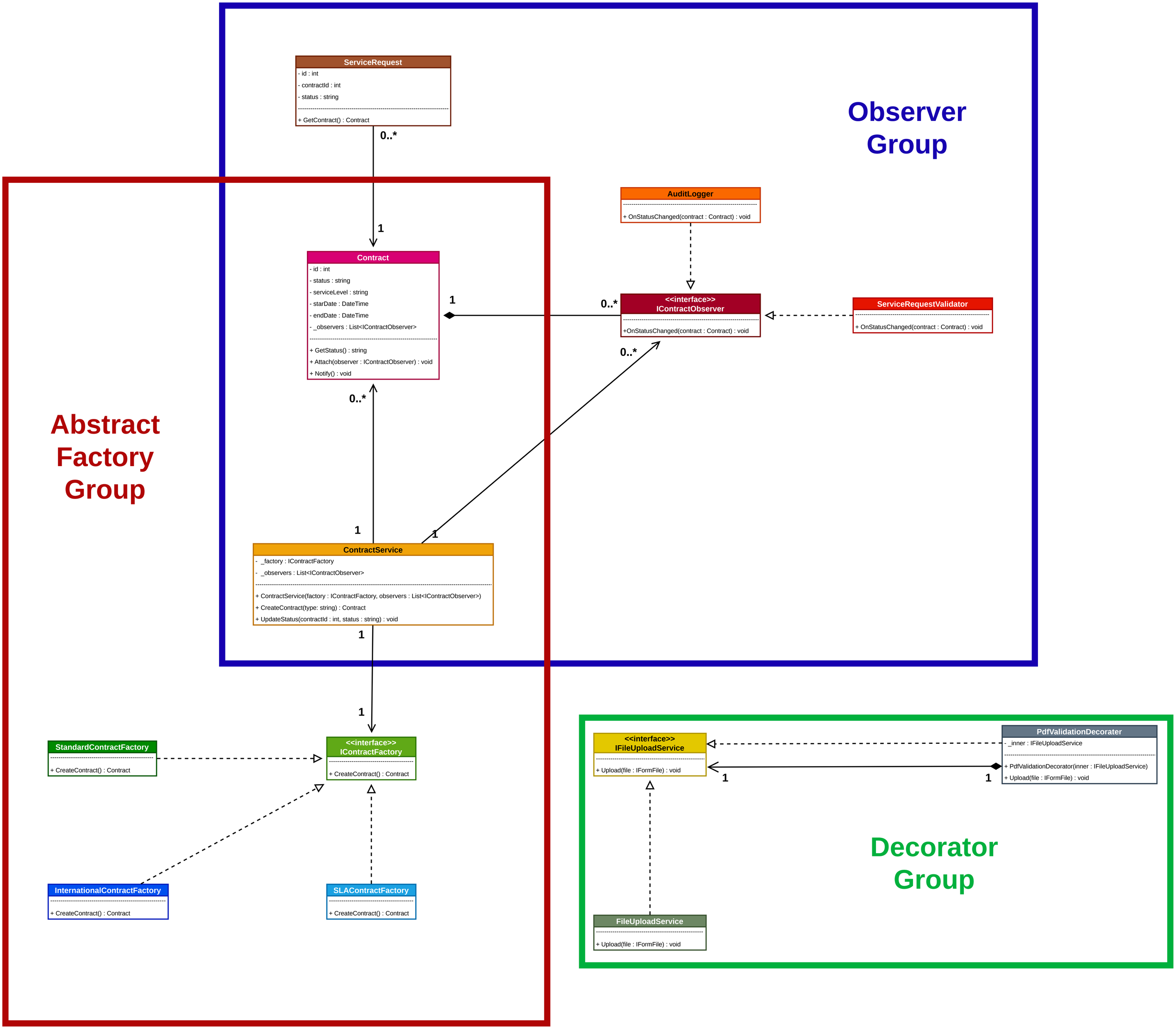
Refactoring Guru. (n.d.-b). *Decorator*. Refactoring Guru. Retrieved March 20, 2026, from https://refactoring.guru/design-patterns/decorator

Swagger. (n.d.). *What Is OpenAPI?* Swagger. Retrieved March 25, 2026, from https://swagger.io/docs/specification/v3_0/about/

Webalert. (2026, January 12). *Uptime SLA Explained: 99.9% vs 99.99% Availability*. Webalert. https://web-alert.io/blog/uptime-sla-explained-99-9-vs-99-99-availability

TOGAF ADM Applied to TechMove GLMS

ADM Phase	Focus Areas	Documents Generated	Techmove Application
A	• Define Scope • Identify Stakeholders	• **Architecture Vision** • **Statement of Architecture Work** • **Stakeholder Register**	• **Vision agreed:** replace the fragmented legacy system (spreadsheets, emails, manual phone calls) with a centralised GLMS • **Scope confirmed:** as contract management, service request processing, international currency conversion, and containerised deployment
B	• **Investigate Current State** • **Determine Goal State** • **Analyse Gap between current and goal**	• **Architecture Definition** • **Gap Analysis** • **Business Capability Map**	• **Current state:** contracts managed in spreadsheets, service requests raised by phone, invoices lost, no enforcement of contract status rules, compliance failures around expired contracts • **Target state:** automated Draft→Active→Expired→On Hold contract lifecycle, ServiceRequests blocked against Expired or On Hold contracts, digitally auditable invoice trail, SLA tracking per client
C	• Data Later Desing • Application Layer Design	• Architecture Definition (updated adding data and applicaiton architecture) • Data and Application Architecture Requirements	• **Data entities:** Client, Contract, ServiceRequest • **Applications:** ASP.NET Core MVC frontend, Cloud SQL Databse, Cloud PDF storage for documents, Live Currency Exchange.
D	• Infrastructure Desing (deployment networking, scaling and security)	• Architecture Definition (updated adding technology architecture) • Technology Architecture Requirements	• **Infrastructure:** Azure App Service, Azure SQl Databse, Azure Filestorage, Currency Exchnage API (ExchangeRate-API, CurrencyLayer or Open Exchange Rates)


Observer Group
ServiceRequest
- id : int
- contractId : int
- status : string
+ GetContract() : Contract
0..*
1
Contract
- id : int
- status : string
- serviceLevel : string
- starDate : DateTime
- endDate : DateTime
- _observers : List<IContractObserver>
+ GetStatus() : string
+ Attach(observer : IContractObserver) : void
+ Notify() : void
AuditLogger
+ OnStatusChanged(contract : Contract) : void
<<interface>>
IContractObserver
+OnStatusChanged(contract : Contract) : void
ServiceRequestValidator
+ OnStatusChanged(contract : Contract) : void
1
0..*
0..*
0..*
Abstract Factory Group
1
1
ContractService
- _factory : IContractFactory
- _observers : List<IContractObserver>
+ ContractService(factory : IContractFactory, observers : List<IContractObserver>)
+ CreateContract(type: string) : Contract
+ UpdateStatus(contractId : int, status : string) : void
1
1
StandardContractFactory
+ CreateContract() : Contract
<<interface>>
IContractFactory
+ CreateContract() : Contract
InternationalContractFactory
+ CreateContract() : Contract
SLAContractFactory
+ CreateContract() : Contract
<<interface>>
IFileUploadService
+ Upload(file : IFormFile) : void
PdfValidationDecorater
- _inner : IFileUploadService
+ PdfValidationDecorator(inner : IFileUploadService)
+ Upload(file : IFormFile) : void
1
1
Decorator Group
FileUploadService
+ Upload(file : IFormFile) : void